



CẨM NANG TOÀN DIỆN CLAUDE TESTING SKILLS

Kiến Trúc, Kỹ Năng & Quy Trình Kiểm Thử Phần Mềm Toàn Trình Hỗ Trợ Bởi AI (End-to-End AI Testing Workflow)

Nội Dung Chính Trong Tài Liệu:

- **Phần I:** Các Kỹ Năng (Skills), MCP Servers và Plugin Cần Thiết
- **Phần II:** Quy Trình Manual Testing Chi Tiết (Kèm Sơ Đồ 8 Chặng & Vòng Lặp Delta)
- **Phần III:** Quy Trình Automation Testing Chi Tiết (Kèm Sơ Đồ 8 Chặng & Auto-Heal)
- **Phần IV:** Các Tiêu Chuẩn Thực Chiến & Quy Tắc Vàng (Quality Gates & Anti-patterns)

Phiên bản: Claude Testing Skills v2.0 (Tối ưu hóa Production-Ready)

Phát triển bởi: AI QA Engineering Team

Môi trường hỗ trợ: Claude Code (CLI / Desktop), Antigravity, VS Code, Cursor

PHẦN I: CÁC SKILL, PLUGIN VÀ MCP CẦN THIẾT

Hệ thống **Claude Testing Skills** được cấu trúc theo mô hình *AI Agent điều phối hướng quy chuẩn (Rule-based & Skill-driven Agent)*. Để AI hoạt động với đầy đủ năng lực, cần thiết lập 3 thành phần nền tảng: **MCP Servers**, **Cấu hình quyền hạn (Permissions/Settings)** và **Thư viện Kỹ năng chuyên biệt (Skills Library)**.

1. Các MCP Servers (Model Context Protocol) Cần Thiết

MCP là cầu nối cho phép AI tương tác trực tiếp với trình duyệt, ứng dụng di động, hệ thống quản lý kiểm thử và API của dự án:

MCP Server	Lệnh cài đặt / Cấu hình	Vai trò & Mục đích sử dụng
Playwright MCP Web Recon & Debug	@playwright/mcp Viewport: 1600x750 File: .mcp.json	Mắt thần của AI trên Web: Cho phép AI mở trình duyệt thật (headed mode), điều hướng, inspect DOM thực tế, chụp ảnh evidence, verify locator chính xác 100% trước khi viết code, và chạy manual test cases tương tác.
Atlassian / Jira MCP Requirements & Bugs	jira-mcp-server API Token + Domain	Đọc User Stories, Acceptance Criteria trực tiếp từ Jira board; tự động đẩy Bug Report sinh từ test fail; đồng bộ kết quả test lên Jira/Xray Test Execution.
Appium MCP Mobile Automation	appium-mcp Port: 4723	Dùng cho kiểm thử ứng dụng Mobile (Native Android, Native iOS, Flutter, Hybrid). AI lấy UI hierarchy thật từ thiết bị/giả lập và nhận diện trạng thái Semantics của Flutter.
Postman / REST MCP API Testing	postman-mcp-server OpenAPI/Swagger	Cho phép AI gọi API thật để kiểm chứng (verification), sinh bộ test contract, kiểm tra 12 status codes và xác thực các lỗ hổng OWASP API Security.

⚠ Quy chuẩn Viewport Headed cho Playwright MCP:

Bắt buộc cấu hình `--viewport-size=1600,750` trong file `.mcp.json` tại thư mục gốc dự án. Tuyệt đối không gọi `browser_resize` vượt kích thước cửa sổ OS vì sẽ làm cắt mất góc bên phải màn hình của tester.

2. Cấu Hình Quyền Hạn & Tối Ưu Môi Trường (`settings.json`)

File `.claude/settings.json` quy định rõ quyền hạn an toàn cho AI Agent trong suốt quá trình làm việc:

- Cho phép (Allow):** Đọc file dự án, đọc biến môi trường `.env` (chứa URL/user test), chạy lệnh `npm`, `npmx`, `mvn`, thực thi Playwright test, inspect DOM qua MCP.
- Nghiêm cấm (Deny):** Tự ý xóa file source code khi chưa có xác nhận từ người dùng; tự động commit hoặc push code lên Git remote; tự cài đặt package global ngoài quyền kiểm soát.
- Quản lý phiên đăng nhập (State Persistence):** Sử dụng file `.auth/user_state.json` (đã gitignore) để lưu session cookie/token, giúp AI bypass màn hình login ở các lượt recon/execution tiếp theo, tránh bị chặn bởi Captcha hoặc Rate-limiting.

3. Hệ Thống 20 Kỹ Năng Chuyên Biệt (Skills Matrix)

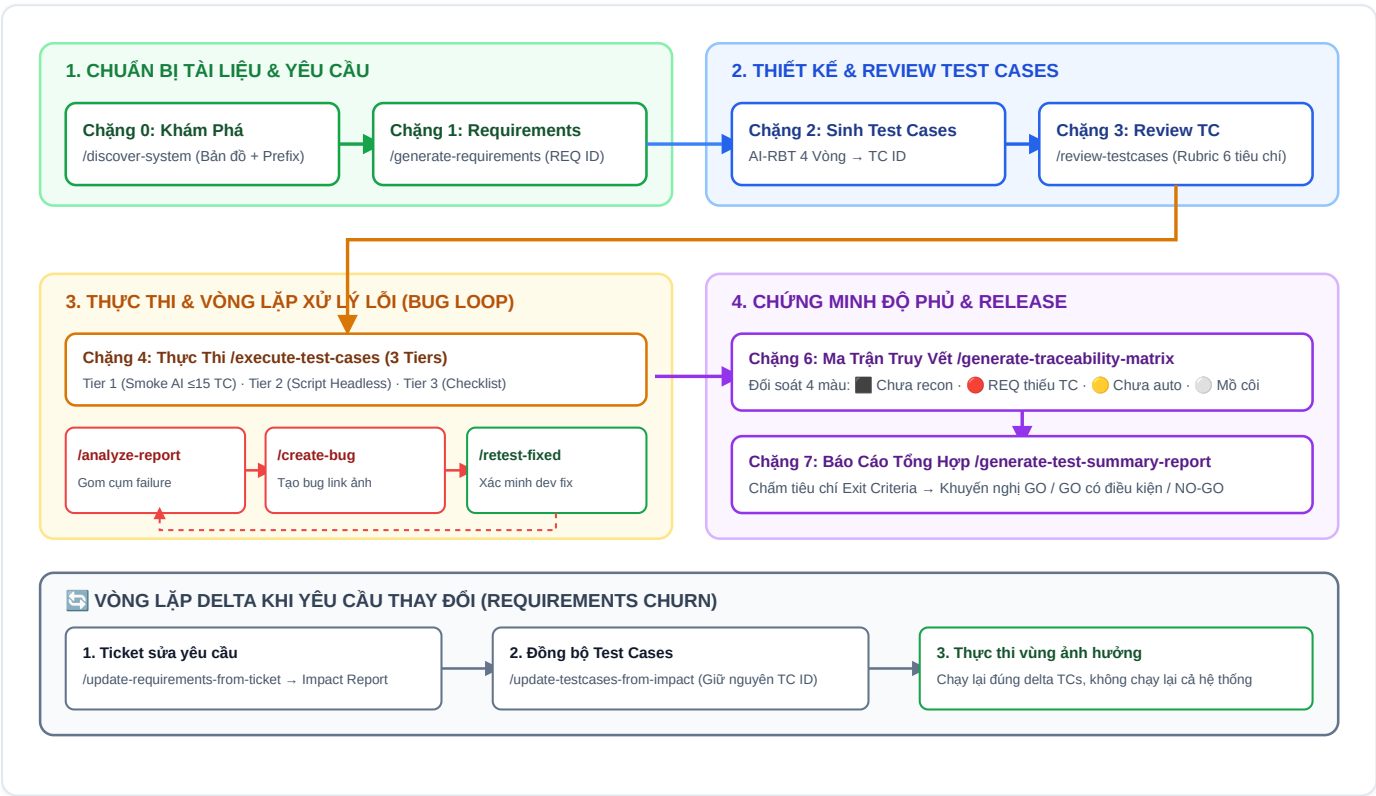
Toàn bộ kỹ năng được phân chia khoa học theo 5 nhóm nghiệp vụ trong chu trình phát triển phần mềm:

Nhóm Nghiệp Vụ	Tên Kỹ Năng (.claude/skills/)	Nhiệm Vụ Cốt Lõi
1. Phân Tích & Yêu Cầu	skills-requirements-analyzer skills-jira-integration skills-user-guide-writer	Khảo sát website/tài liệu, bóc tách `REQ ID`, phát hiện mơ hồ (Ambiguity), xuất tài liệu đặc tả chuẩn và viết hướng dẫn sử dụng (User Guide) theo chuẩn ISO/IEC/IEEE 26514.
2. Manual Testing (AI-RBT)	skills-rbt-manual-testing skills-manual-test-executor skills-testcase-reviewer	Sinh Manual TC theo 4 chế độ (QUICK, FULL RBT 6 bước, CHECKLIST, DELTA), phân loại 4 vòng kiểm thử, review rubric 6 tiêu chí và điều khiển browser thực thi test có chụp ảnh bằng chứng.
3. Automation Engineering	skills-qa-automation-engineer skills-framework-architect skills-automation-code-reviewer skills-mobile-debug-agent	Master orchestrator cho automation; scaffold framework POM; review code chống pass giả và hard sleep; tự động nhận diện Flutter semantics và Native mobile hierarchy.
4. Tự Sửa & Ổn Định	skills-smart-locator-agent skills-locator-healer-agent skills-flaky-test-analyzer	Sinh locator semantic theo vai trò người dùng (`getByRole`, `getByLabel`); tự động quét và sửa locator khi UI thay đổi; phân tích nguyên nhân gốc khiến test lúc pass lúc fail.
5. Dữ Liệu & Báo Cáo	skills-test-data-generator skills-coverage-traceability skills-test-report-analyzer skills-test-summary-reporter skills-bug-reporter	Sinh test data ngẫu nhiên có truy vết (`prefix_test_ts`); lập ma trận RTM 3 tầng; phân tích gom cụm lỗi; tạo bug report chuẩn link ảnh; lập báo cáo Test Summary tổng hợp mốc release (GO/NO-GO).

PHẦN II: QUY TRÌNH MANUAL TESTING CHI TIẾT

Quy trình Manual Testing gồm 8 chặng khép kín (Chặng 0 đến Chặng 7). Mất xích sống còn giữ toàn bộ chuỗi là mã REQ ID (sinh ra ở Chặng 1) gắn trực tiếp vào cột REQ ID của từng Test Case (Chặng 2), làm cơ sở cho Ma trận truy vết (Chặng 6) và Báo cáo Release (Chặng 7).

1. Sơ Đồ Toàn Cảnh 8 Chặng Manual Testing



2. Chi Tiết Từng Chặng & Quy Tắc Thực Hiện

Chặng 0: Khám Phá Hệ Thống (`/discover-system`)

Chạy 1 lần duy nhất cho toàn bộ hệ thống. Trả lời câu hỏi cốt lõi: "Hệ thống có những module nào và tiền tố phân định là gì?"

- **3 Chế độ linh hoạt:** **UI** (chỉ có URL web), **HYBRID** (có URL + một phần tài liệu), **DOC** (chưa có account, chỉ có tài liệu spec/Jira).
- **Đầu ra:** `docs/requirements/README.md` (danh mục toàn hệ thống, gán prefix duy nhất cho từng module như `CUST`, `ORD`) và `_discovery/system_map.md` (bản đồ cây điều hướng).
- **Checkpoint dừng:** AI dừng hỏi tester để làm rõ các module bị ẩn sâu trong menu, trang phân quyền cao hoặc các vùng URL đặc biệt.

Chặng 1: Sinh Requirements & Cổng Tự Soát 10 Tiêu Chí

Chạy lặp theo từng module bằng `/generate-requirements-from-website`. AI mở từng form, kích hoạt validation để lấy thông báo lỗi nguyên văn, bắt request qua Network và chụp ảnh evidence full-page.

- Đánh mã chuẩn mực: `REQ-[PREFIX]-01`.
- **Cổng tự soát REQ (10 tiêu chí):** Tên REQ và AC phải cùng mệnh đề; khẳng định phủ định phải có phép thử trạng thái sạch 4 bước; AC không chứa token/id động theo phiên; 1 REQ = 1 rule độc lập; tồn tại trong DOM không đồng nghĩa người dùng dùng được.

Chặng 2: Sinh Test Cases Theo Khung 4 Vòng & AI-RBT

Áp dụng phương pháp Kiểm thử dựa trên rủi ro (Risk-Based Testing). Bộ TC sinh theo thứ tự bắt buộc **4 Vòng**:

1. **Vòng 1 - Smoke:** Màn hình dựng được, luồng sống còn không vỡ. Bắt buộc có nhánh *UI cơ bản* (nhấn nguyên văn, thứ tự field, trạng thái mặc định, con trỏ).
2. **Vòng 2 - Functional:** Thân chính của bộ TC (~60-70% số lượng). Tra cứu và bao phủ đầy đủ **15 loại Input Field Types**, Component checklist (Table, CRUD, Modal, Toast) và kỹ thuật thiết kế (EP, BVA, Decision Table, State Transition).
3. **Vòng 3 - Technical:** Các kiểm tra kỹ thuật không nhìn thấy bằng mắt (API status codes, Database soft/hard delete, audit logs) gắn tag `@TechCheck`.
4. **Vòng 4 - Non-functional:** Rà soát theo 9 đặc tính **ISO/IEC 25010:2023** (Security, A11y, Performance thô, Network resilience). Nếu không kiểm phải ghi rõ **— Ngoài phạm vi** kèm người chịu trách nhiệm.

Chặng 3: Review Test Cases (`/review-testcases`)

Chấm chất lượng bộ TC theo **Rubric 6 tiêu chí** (Tính độc lập, Tính khả thi, Độ bao phủ biên, Dữ liệu cụ thể, Expected 1-1 với Step, Chuẩn Metadata). Có 2 chế độ: `REVIEW` (chỉ ra điểm yếu) và `FIX` (tự động xuất file bản cải thiện mà vẫn giữ nguyên file gốc để đối chiếu).

Chặng 4: Thực Thi Test Manual & Chiến Lược 3 Tầng (Execution Tiers)

🎯 **Chiến lược phân tầng thực thi (Execution Tiers) tối ưu thời gian & token:**

- **Tier 1 - AI-Assisted Interactive:** Dùng `/execute-test-cases` cho các suite nhỏ (≤ 15 TC thuộc Vòng 1 Smoke & P1 Critical flow). AI mở browser thật qua Playwright MCP, đi từng bước và chấm PASS/FAIL.
- **Tier 2 - Headless Automation Batch:** Với suite lớn (> 15 TC về validation form, boundary, logic), chuyển sang sinh script qua `/generate-automation-from-testcases` và chạy headless (50 TC chạy trong < 1 phút thay vì 60 phút click tay).
- **Tier 3 - Checklist Human-in-the-loop:** Với các case visual, responsive, trải nghiệm, xuất file checklist qua `/generate-checklist-test` để người dùng tick tay trực tiếp trên Web Viewer.

Chặng 5: Xử Lý Lỗi & Vòng Lặp Bug Khép Kín

Khi có nhiều test FAIL, quy tắc bắt buộc là **không tạo bug tràn lan**:

- Chạy `/analyze-test-report` trước để gom cụm failure theo nguyên nhân gốc (10 TC fail thường chỉ bắt nguồn từ 1-2 bug).
- Chạy `/create-bug-report` : Tự động trở link tới ảnh evidence trong thư mục execution tương ứng, không nhân bản ảnh.
- Sau khi Dev fix, chạy `/retest-fixed-bugs` : Chạy lại kịch bản lỗi (chấm `FIXED` , `NOT_FIXED` , `PARTIAL`) kèm **kiểm thử hồi quy (Regression)** quanh vùng ảnh hưởng.

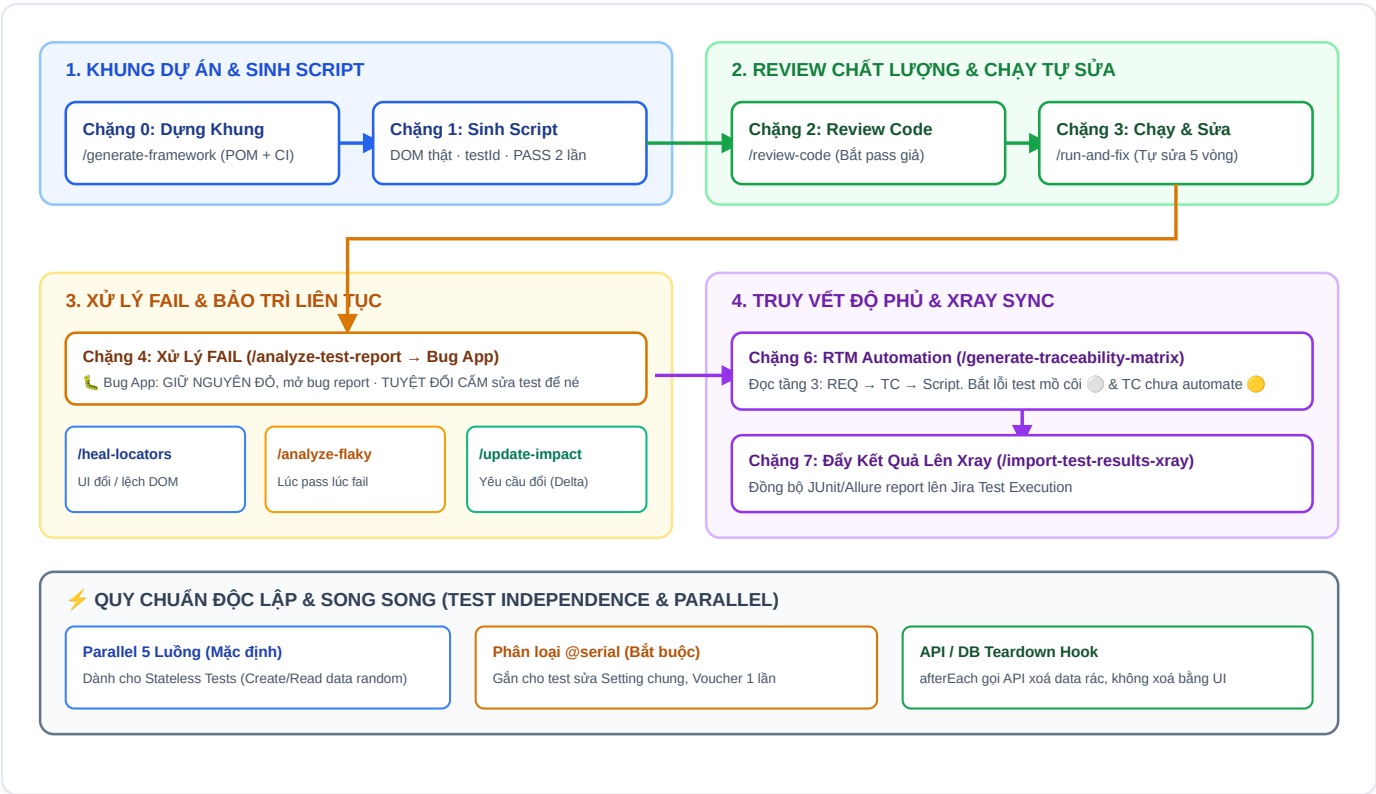
Chặng 6 & 7: Ma Trận Truy Vết (RTM) & Báo Cáo Tổng Hợp (Test Summary)

- **RTM** (`/generate-traceability-matrix``): Chỉ ra 4 nhóm:  Module chưa có tài liệu,  REQ chưa có TC,  TC chưa automate,  Test mờ côi không map về REQ nào.
- **Test Summary** (`/generate-test-summary-report``): Tổng hợp toàn diện đợt release, đối chiếu với tiêu chí Exit Criteria và đưa ra khuyến nghị rõ ràng: **GO**, **GO có điều kiện** hoặc **NO-GO**.

PHẦN III: QUY TRÌNH AUTOMATION TESTING CHI TIẾT

Quy trình Automation Testing nối tiếp từ bộ Manual Test Cases đã review ở Chặng 3 của Flow Manual. **Mắt xích liên kết giữa hai thế giới là mã `TC ID`.** Trong mã nguồn script, mỗi test bắt buộc phải gắn nhãn `allure.label('testId', 'CRM_LOGIN_TC_001')` để hệ thống tự động map vào RTM tầng 3.

1. Sơ Đồ Toàn Cảnh 8 Chặng Automation Testing



2. Chi Tiết Từng Chặng & Các Quy Tắc Vàng Automation

Chặng 0: Dựng Khung Automation Framework (`/generate-automation-framework`)

Chạy 1 lần cho cả dự án (bỏ qua nếu dự án đã có framework). AI dựng cấu trúc chuẩn:

- Mô hình **Page Object Model (POM)**: Tách biệt rõ `pages/`, `tests/` và `utils/`.
- Allure Report tích hợp cục bộ**: Chạy `npm run report` mở được ngay trên máy chưa từng cài đặt Allure.
- CI/CD GitHub Actions**: Có sẵn workflow headless chạy parallel trên runner.

Chặng 1: Sinh Test Scripts Theo 6 Bước Chuẩn Xác

- Parse TC**: Đọc bảng manual TC đã review, lấy tiền tố, precondition và các bước Arrange/Act/Assert.
- Inspect DOM thực tế**: Mở browser qua Playwright MCP để thu thập locator semantic (`getByRole`, `getByLabel`). **TUYỆT ĐỐI CẤM ĐOÁN LOCATOR**.
- Thiết kế Page Class**: Gom locators và action methods vào Page Object.

4. **Sinh Test Data động:** Format bắt buộc `[prefix]_[testName]_[timestamp]_[random]` .
5. **Viết Test Class:** Gắn nhãn `allure.label('testId', TC_ID)` , chia step Tiếng Việt rõ ràng.
6. **Chạy & Auto-heal:** Test bắt buộc phải **PASS 2 lần liên tiếp** mới được bàn giao.

Chặng 2: Review Code Automation (`/review-automation-code`)

Bắt các lỗi tiềm ẩn mà mắt thường không thấy: **Pass giả (Assertion yếu)**, hard sleep (`waitForTimeout` , `Thread.sleep`), locator nhét inline trong file test, thiếu screenshot cuối test.

Chặng 3: Chạy Suite & Tự Sửa Lỗi (`/run-and-fix-tests`)



LUẬT BẤT BIẾN QUAN TRỌNG NHẤT:

CẤM sửa test cho xanh khi nguyên nhân là **BUG CỦA ỨNG DỤNG!**

Tuyệt đối không nói lỏng assertion, không sửa expected theo actual sai, không gắn tag `.skip`` để giấu lỗi. Khi xác định là Bug App, bắt buộc giữ nguyên trạng thái ĐỎ và chuyển sang `/create-bug-report` .

Chặng 5: Ba Nhánh Tín Hiệu Bảo Trì (Maintenance)

Tín hiệu thực tế	Lệnh xử lý	Cách hoạt động
UI vừa deploy bản mới, gây locator hàng loạt	<code>/heal-locators</code>	Quét DOM thật ở chế độ headed, phát hiện locator gãy, sắp gãy hoặc match sai element; cập nhật lại Page Object.
Test lúc pass lúc fail, code không đổi	<code>/analyze-flaky-tests</code>	Phân tích Race condition, thiếu smart wait, xung đột test data do chạy song song; tối ưu lại wait strategy.
Requirements thay đổi sau ticket mới	<code>/update-automation-from-impact</code>	Chỉ sửa đúng những dòng script tương ứng với Delta TC List. Cấm sinh lại toàn bộ file làm mất các fix tích lũy.

PHẦN IV: ĐIỀU PHỐI ĐA ĐẶC VỤ & BẢNG TRA NHANH

1. Kiến Trúc Điều Phối Đa Đặc Vụ (Multi-Agent Subagent Delegation)

Khi dự án mở rộng lên quy mô lớn (từ 3 module trở lên), việc chạy tuần tự trên 1 phiên chat duy nhất sẽ làm tràn context window và tăng chi phí token. Quy chuẩn điều phối khuyến nghị:

Vai trò	Không gian Context	Nhiệm vụ cụ thể
Orchestrator Agent (Phiên chính)	Toàn cục dự án (~8.000 tokens)	Chạy <code>/discover-system</code> , quản lý file danh mục <code>docs/requirements/README.md</code> và <code>system_map.md</code> ; phân chia nhiệm vụ và giám sát tiến độ của các Subagents.
Worker Subagent 1..N (Phiên song song)	Độc lập theo Module (Context sạch 100%)	Mỗi Subagent nhận 1 module: tự động chạy Chặng 1 (Requirements) và Chặng 2 (Sinh Test Cases); ghi file trực tiếp vào <code>docs/requirements/<module>/</code> và <code>docs/testcases/<module>/</code> .

2. Bảng Tra Nhanh 35 Slash Commands & 32 Prompt Mẫu

Mã	Slash Command	File Prompt Mẫu	Giai đoạn & Mục đích
00	<code>/discover-system</code>	<code>prompt_00_discover_system.txt</code>	Khám phá hệ thống chưa có tài liệu
01	<code>/generate-requirements-from-website</code>	<code>prompt_01_generate_requirements.txt</code>	Recon UI sinh đặc tả yêu cầu & REQ ID
02	<code>/generate-testcases-manual-rbt</code>	<code>prompt_02_generate_test_cases.txt</code>	Sinh Manual TC 4 Vòng chuẩn AI-RBT
03	<code>/generate-automation-framework</code>	<code>prompt_03_create_framework_*.txt</code>	Dựng framework Playwright / Selenium / Appium
04	<code>/generate-automation-from-testcases</code>	<code>prompt_04_generate_script_*.txt</code>	Sinh test script tự động chuẩn POM
05	<code>/generate-automation-from-testcases</code>	<code>prompt_05_convert_manual_to_auto.txt</code>	Chuyển đổi Manual TC sang Automation
06	<code>/review-automation-code</code>	<code>prompt_06_review_automation_code.txt</code>	Review code tự động theo DoD
07	<code>/generate-test-data</code>	<code>prompt_07_generate_test_data.txt</code>	Sinh test data ngẫu nhiên có truy vết
08	<code>/analyze-flaky-tests</code>	<code>prompt_08_analyze_flaky_tests.txt</code>	Tìm nguyên nhân gốc và trị flaky tests
09	<code>/generate-api-tests-from-swagger</code>	<code>prompt_09_generate_api_tests.txt</code>	Sinh API test 12 status code & OWASP
10	<code>/create-bug-report</code>	<code>prompt_10_create_bug_report.txt</code>	Tạo bug report chuẩn link evidence
11	<code>/analyze-test-report</code>	<code>prompt_11_analyze_test_report.txt</code>	Gom cụm failure theo root cause
12	<code>/review-testcases</code>	<code>prompt_12_review_testcases.txt</code>	Review chất lượng bộ

Mã	Slash Command	File Prompt Mẫu	Giai đoạn & Mục đích
			Manual TC
13	/generate-traceability-matrix	prompt_13_generate_traceability_matrix.txt	Lập ma trận truy vết RTM 3 tầng
14	/generate-api-mocks	prompt_14_generate_api_mockes.txt	Sinh API mocks giả lập lỗi 500/timeout
15	/generate-checklist-test	prompt_15_generate_checklist.txt	Sinh checklist tick tay cho smoke/release
16	/execute-test-cases	prompt_16_execute_test_cases.txt	Thực thi manual TC có chụp ảnh evidence
17	/run-and-fix-tests	prompt_17_run_and_fix_tests.txt	Chạy suite có sẵn & tự sửa lỗi test
18	/heal-locators	prompt_18_heal_locators.txt	Tự phục hồi locator hỏng sau khi đổi UI
19	/update-automation-from-impact	prompt_19_update_automation_impact.txt	Cập nhật script theo Impact Report (Delta)
21	/update-requirements-from-ticket	prompt_21_update_requirements.txt	Cập nhật đặc tả khi ticket đổi yêu cầu
23	/retest-fixed-bugs	prompt_23_retest_fixed_bugs.txt	Retest bug đã fix + regression quanh vùng fix
24	/generate-test-summary-report	prompt_24_generate_summary_report.txt	Báo cáo tổng hợp mốc release (GO/NO-GO)
27	/update-testcases-from-impact	prompt_27_update_testcases_impact.txt	Đồng bộ Test Cases theo Impact Report